

1. Características de GNU/Linux:

a. Mencione y explique las características más relevantes de GNU/Linux.

GNU/Linux destaca por poseer un conjunto de características técnicas y de licenciamiento que lo definen como un sistema robusto y flexible:

Multiusuario: Permite que múltiples usuarios accedan y utilicen los recursos del sistema al mismo tiempo aislando los entornos y datos de cada uno

Estabilidad y seguridad: Su estabilidad y robustez lo hacen muy confiable para servidores y grandes sistemas

Multitarea y multiprocesador: Tiene la capacidad de ejecutar de forma concurrente numerosos procesos y cuenta con soporte nativo para múltiples procesadores

Portable: Funciona en arquitecturas de hardware muy diversas

Open Source: los usuarios pueden ver su código, estudiarlo, modificarlo, adaptarlo o compartirlo sin problemas

Todo es un archivo: Se tratan prácticamente todos los elementos como si fueran archivos

Particionado flexible: permite que las diferentes ramas del sistema de archivos se encuentren en particiones de disco o dispositivos independientes

Case sensitive: distingue entre mayus y minus (por ejemplo en comandos, carpetas, rutas)

Múltiples intérpretes de comandos: Diversas interfaces de línea de comandos que facilitan la interacción con el OS y son altamente programables a través de scripts

b. Mencione otros sistemas operativos y compárelos con GNU/Linux en cuanto a los puntos mencionados en el inciso a.

GNU/Linux es open source, en cambio Windows y macOS son sistemas propietarios con un costo asociado, no permiten su distribución, su código fuente está oculto y no puede ser modificado

La diferencia en la consola es que Linux permite tener un historial entre sesiones, además de ser una parte muy importante para el sistema. macOS recientemente adoptó una consola más

avanzada, pero windows tiene una linea de comandos menos integrada

En estabilidad y seguridad: windows es menos seguro y de principales objetivos de los atacantes por su debilidad en este punto, en estabilidad es sabido que es inconsistente y propenso a fallos. Mac OS en cambio es muy estable ya que esta bien integrado a su hardware propietario y tienen un gran enfoque en la seguridad

c. ¿Qué es GNU?

Gnu es un acronimo que significa GNU no es unix. Es un proyecto de sistema operativo similar a unix constituido totalmente por software libre. Su marco etico define las 4 libertades fundamentales del software: libertad de usar el programa para cualquier proposito, estudiar su funcionamiento, distribuir copias y publicar mejoras para beneficiar a la comunidad

d. Indique una breve historia sobre la evolución del proyecto GNU.

El proyecto GNU, iniciado por Richard Stallman en 1983, buscaba crear un sistema operativo tipo Unix completamente libre. Para garantizar la libertad del software, se creó la Licencia Pública General de GNU (GPL). En 1985, Stallman fundó la Free Software Foundation (FSF) para financiar el proyecto.

Para 1990, GNU ya contaba con varias herramientas importantes, como el editor Emacs y el compilador GCC, pero le faltaba un núcleo (kernel). El primer intento de crear un núcleo, llamado TRIX, fue abandonado debido a su complejidad. Luego, se intentó desarrollar GNU Hurd basado en el núcleo MACH, pero también fracasó.

En 1991, Linus Torvalds creó el núcleo Linux y lo distribuyó bajo la GPL. En 1992, Torvalds y Stallman decidieron combinar el núcleo Linux con los componentes de GNU, dando lugar al sistema operativo conocido como GNU/Linux, que sigue siendo un ejemplo destacado de software libre.

e. Explique qué es la multitarea, e indique si GNU/Linux hace uso de ella.

La multitarea es la capacidad de un sistema operativo de ejecutar y gestionar concurrentemente o de manera simultanea multiples procesos o programas, distribuyendo sus recursos entre ellos de manera ordenada

Si, hace uso de ella, es uno de sus puntos fuertes. El kernel administra el tiempo de procesamiento de la cpu de manera dinamica entre los programas activos

f. ¿Qué es POSIX?

Posix (Portable Operating System Interface) es una norma creada por el IEEE que define cómo deben comunicarse los programas con los sistemas operativos. Permite que el código y los scripts funcionen de forma idéntica en varios sistemas tipo Unix, como Linux o macOS

2. Distribuciones de GNU/Linux:

a. ¿Qué es una distribución de GNU/Linux? Nombre al menos 4 distribuciones de GNU/Linux y cite diferencias básicas entre ellas.

Una "distro" de linux es una variante del OS que incluye el kernel de linux y una coleccion de software. Cada distribucion tiene su manera de manejar los paquetes, las actualizaciones, los archivos, entorno grafico y sus configuraciones iniciales.

Arch linux es una distribucion para usuarios avanzados que buscan tener un mayor control de lo que se encuentra en su computadora

Linux mint cuenta con una interfaz mas amigable para nuevos usuarios (escritorio similar al windows)

Ubuntu es accesible para principiantes y altamente utilizada en escritorios y servidores

Fedora esta enfocado en tener software actualizado y descata por estar siempre a la vanguardia con actualizaciones constantes

Debian es conocida por su estabilidad y robustes, ideal para servidores y sistemas criticos

b. ¿En qué se diferencia una distribución de otra?

Las distribuciones se diferencian en el fin para el que son usadas, hay distribuciones orientadas a programacion, a ser servidores (sin interfaz), a usuarios nuevos(interfaz comoda), a usuarios avanzados, gaming etc

c. ¿Qué es Debian? Acceda al sitio <https://www.debian.org/> e indique cuáles son los objetivos del proyecto y una breve cronología del mismo.

Debian es un sistema operativo libre y gratuito basado en Linux, desarrollado por una comunidad global de voluntarios. Es famoso por su gran estabilidad, su alta seguridad y por ser la base de muchos otros sistemas populares como Ubuntu
su principal objetivo es crear un OS libre disponible para todo el mundo, ademas de ofrecer estabilidad y fiabilidad, funcionar en una amplia variedad de arquitecturas de software, aprender mas sobre las computadoras y ayudar a los demas mejorando y contribuyendo al software libre

1993: Ian Murdock anuncia el proyecto Debian, inspirándose en la necesidad de una distribución que sea mantenida abierta y democráticamente.

1994: Se lanza la primera versión oficial de Debian.

1996: El proyecto crece y se introduce el "Debian Social Contract", que define los principios y objetivos del proyecto.

1997: Se publica Debian 1.3, la última versión que incluye el núcleo original de Linux.

2000: Debian introduce apt-get, que simplifica la gestión de paquetes y actualizaciones.

2015: Debian lanza Debian 8 "Jessie", que incluye systemd como sistema de inicio predeterminado.

Actualidad: Debian continúa siendo una distribución fundamental que sirve de base para muchas otras distribuciones, incluidas Ubuntu y Linux Mint.

3. Estructura de GNU/Linux:

a. Nombre cuáles son los componentes fundamentales de GNU/Linux.

Kernel(nucleo)

Shell(interprete de comandos)

FileSystem(sistema de archivos)

b. Mencione y explique la estructura básica del Sistema Operativo GNU/Linux.

Hardware: Capa física.

Kernel: Gestión de recursos y comunicación con el hardware.

Bibliotecas del sistema y llamadas al sistema: Soporte para la interacción de las aplicaciones con el kernel.

Shell: Interfaz de usuario para interactuar con el sistema.

Aplicaciones: Software que se ejecuta en el entorno del usuario.

4. Kernel:

a. ¿Cuáles son sus funciones principales?

En un sentido estricto, el kernel es el sistema operativo en sí mismo. Sus funciones principales son:

Administración de memoria: Controla la asignación y el uso de la memoria física y virtual (RAM + SWAP).

Administración de la CPU: Gestiona la planificación y ejecución de los procesos y programas.

Administración de dispositivos de Entrada/Salida (E/S): Coordina la interacción segura de los controladores de dispositivos (*device drivers*) con el hardware

Gestión de la Red: El kernel facilita la comunicación en red a través de la gestión de protocolos

de red y el control de acceso a interfaces de red. Esto incluye la configuración de la pila TCP/IP, enrutamiento y seguridad de red

Seguridad y Control de Acceso: El kernel proporciona mecanismos de seguridad que controlan el acceso de los usuarios y procesos a los recursos del sistema. Esto incluye la gestión de permisos, autenticación y control de acceso a nivel de procesos

b. ¿Es posible tener más de un Kernel de GNU/Linux instalado en la misma máquina?(ayuda /boot)

Sí, es perfectamente posible. Los archivos binarios de los diferentes kernels se guardan en el sistema y, durante el arranque, el gestor de arranque (bootloader) como GRUB lee estos archivos de configuración y presenta un menú interactivo al usuario. Esto permite seleccionar de forma manual qué versión o imagen de kernel específica se desea cargar en la memoria para iniciar el sistema operativo

c. ¿En qué directorio se encuentra ubicado?

Las imágenes binarias del Kernel (utilizadas para el arranque del sistema, como vmlinuz-A.B.C) se encuentran ubicadas en el directorio /boot (o en la partición destinada a /boot en el disco)

5. **Intérprete de comandos (Shell):**

a. ¿Qué es y cuáles son sus funciones?

Es un programa que funciona como una interfaz de línea de comandos en modo texto, permitiendo la comunicación directa entre el usuario y el núcleo del sistema operativo.

Funciones principales:

1. Leer y analizar las órdenes de texto ingresadas por el usuario a través del teclado cuando se muestra el *prompt*.
2. Interpretar y traducir dichos comandos en instrucciones comprensibles para el sistema operativo.
3. Entregar las instrucciones al kernel para que se ejecuten los programas correspondientes.
4. Mostrar las respuestas y salidas de texto de los comandos en la misma ventana de la terminal.
5. Permitir la automatización de tareas mediante la programación de scripts (es una herramienta programable)

b. Investigue y mencione al menos 3 intérpretes de comandos que existen para GNU/Linux y compárelos entre ellos.

Bash: Es el intérprete de comandos predeterminado en la mayoría de las distribuciones de GNU/Linux.

Características:

Compatibilidad: Compatible con scripts escritos para el Bourne Shell (sh) y ofrece muchas mejoras.

Scripting Avanzado: Soporte para programación avanzada, incluyendo bucles, condicionales y funciones.

Zsh: Es un shell interactivo y de scripting que ofrece una serie de características avanzadas no presentes en otros shells.

Características: Completado Automático Mejorado: Completado de comandos y nombres de archivos más inteligente y flexible, con opciones de autocompletado extensas.

Corrección de Comandos: Capacidad para corregir automáticamente errores en los comandos escritos.

Fish: Es un shell diseñado para ser amigable y fácil de usar, con un enfoque en la facilidad de uso y la interacción amigable.

Características: Sintaxis Amigable: La sintaxis de Fish es más fácil de aprender y usar, con características de scripting más intuitivas. Interfaz de Usuario: Ofrece una interfaz interactiva intuitiva y visualmente atractiva.

c. ¿Dónde se ubican (path) los comandos propios y externos al Shell?

Comandos propios (internos): No poseen un archivo ejecutable en el disco y, por tanto, no tienen un *path* físico externo. Están programados dentro del propio código de la shell en ejecución, la cual los reconoce y ejecuta directamente sin ayuda externa (por ejemplo: cd, export, exit).

Comandos externos: Son archivos ejecutables independientes almacenados físicamente en el disco. Cuando el shell recibe un comando que no es interno, lo busca secuencialmente en las rutas de directorios especificadas en la variable de entorno PATH (consultable mediante echo \$PATH). Estos ejecutables se ubican típicamente en directorios como /bin, /usr/bin o /usr/local/bin

d. ¿Por qué considera que el Shell no es parte del Kernel de GNU/Linux?

El shell no es parte del kernel porque se ejecuta en el espacio de usuario (user space), mientras que el kernel opera en el espacio de kernel (kernel space). Esta separación es fundamental en la arquitectura de los sistemas operativos modernos:

El kernel gestiona directamente el hardware y los recursos del sistema, mientras que el shell proporciona una interfaz para que los usuarios interactúen con el sistema operativo sin involucrarse directamente con las operaciones a nivel de kernel.

e. ¿Es posible definir uno distinto para cada usuario?

Sí, es completamente posible. El sistema operativo permite asignar a cada usuario un intérprete de comandos por defecto que se iniciará automáticamente cada vez que inicie sesión (*login*) en una terminal. Esta asignación personalizada se registra de manera explícita en el archivo de configuración `/etc/passwd`

6. El sistema de Archivos (File System) en Linux:

a. ¿Qué es?

Un sistema de archivos (File System) es la estructura y el método que utiliza un sistema operativo para almacenar, organizar y gestionar los archivos en un dispositivo de almacenamiento, como un disco duro o una unidad SSD. Un sistema de archivos define cómo se guardan los datos en el dispositivo, cómo se organizan en directorios y cómo se accede a ellos mediante nombres de archivo.

b. ¿Cuál es la estructura básica de los File System en GNU/Linux? Mencione los directorios más importantes e indique qué tipo de información se encuentra en ellos. ¿A qué hace referencia la sigla FHS?

a sigla FHS: Significa Filesystem Hierarchy Standard (Estándar de Jerarquía del Sistema de Archivos). Es un estándar creado en 1994 con el fin de unificar y normalizar la ubicación y el contenido de los directorios clave dentro de las distribuciones de GNU/Linux y sistemas de la familia Unix.

Directorios más importantes según la FHS:

- `/` (Raíz): El nivel supremo que encabeza toda la estructura jerárquica de archivos.
- `/bin`: Contiene archivos binarios y programas ejecutables de uso común indispensables para que el sistema funcione (como `ls`, `cp`, `mv`).
- `/sbin`: Contiene ejecutables y herramientas del sistema esenciales destinados al uso exclusivo del administrador del sistema (*root*).
- `/dev`: Contiene enlaces especiales que representan los dispositivos de hardware físicos del sistema (discos, terminales, puertos).
- `/etc`: Directorio donde se almacenan los archivos de configuración del sistema y de los servicios instalados (por ejemplo, `/etc/passwd`).
- `/home`: Espacio destinado a alojar los directorios personales de los usuarios del sistema.
- `/root`: Es el directorio personal del súper usuario o administrador de sistema (*root*).
- `/lib`: Almacena las imágenes de librerías compartidas esenciales que los programas ejecutables cargan en memoria para evitar duplicar código.
- `/usr`: Contiene subdirectorios con programas, librerías, documentación y recursos opcionales de usuario (como `/usr/bin`, `/usr/include` con cabeceras de C, o `/usr/src` con códigos fuente).
- `/var`: Aloja archivos de datos que varían de tamaño continuamente o que tienden a crecer, como registros de eventos (*logs* en `/var/log`) o colas de impresión y correo (`/var/spool`).
- `/proc`: Es un sistema de archivos virtual cuyos ficheros existen en la memoria RAM y sirven para consultar información directa de los procesos activos.

- /tmp: Destinado a almacenar archivos temporales creados por el sistema o las aplicaciones durante su ejecución

c. Mencione sistemas de archivos soportados por GNU/Linux.

ext4 (Fourth Extended Filesystem): Ofrece buenas prestaciones en términos de rendimiento y confiabilidad, además de soporte para grandes volúmenes de datos.

Btrfs (B-tree File System): Conocido por sus capacidades avanzadas, como la creación de instantáneas (snapshots), compresión de datos, y mejoras en la gestión de volúmenes.

XFS: Diseñado para escalabilidad y rendimiento en sistemas de alto rendimiento, como servidores. Es altamente eficiente en la manipulación de archivos grandes y cargas de trabajo intensivas.

7. Particiones:

a. Definición. Tipos de particiones. Ventajas y Desventajas.

- Definición: Una partición es una forma de dividir lógicamente un disco duro físico en secciones independientes.
- Tipos de particiones:
 - Partición primaria: Es una división física básica o "cruda" del disco. Debido al espacio limitado en la tabla de particiones del MBR, solo se permite crear un máximo de 4 particiones primarias por disco, y su información estructural se almacena directamente en dicho sector de arranque.
 - Partición extendida: Es una partición especial diseñada para superar el límite de 4 particiones físicas. Solo puede existir una partición extendida por disco y no se le define un sistema de archivos de forma directa, ya que su única función es servir de contenedor para subparticiones.
 - Partición lógica: Es un volumen que se crea ocupando una parte o la totalidad del espacio de la partición extendida. Se les puede asignar cualquier sistema de archivos y se estructuran internamente conectándose entre sí como una lista enlazada.
- Ventajas del particionamiento:
 - Permite instalar múltiples sistemas operativos en particiones separadas dentro de una misma computadora.
 - Facilita formatear cada sección del disco con un tipo de sistema de archivos diferente según la necesidad.
 - Es una buena práctica de administración para separar los datos del usuario de las aplicaciones y del sistema operativo.
 - Permite reservar un espacio exclusivo para tener una partición de restauración (*restore*) de todo el sistema.
 - Brinda seguridad al permitir ubicar el kernel en una partición configurada como solo lectura o que ni siquiera se monte para los usuarios.
 - Ayuda a aislar directorios del sistema operativo que crecen constantemente (como /tmp o /var), evitando que afecten a otras partes del disco.

- Permite superar las limitaciones de sistemas antiguos (como DOS o Windows 95) que no podían gestionar sistemas de archivos mayores a 2 GB.
- Desventajas:
 - Particionar de forma excesiva el disco duro puede traer inconvenientes prácticos, tales como la rigidez en la administración del espacio (p. ej., quedarse sin almacenamiento en una partición pequeña mientras que otra queda vacía) o el desperdicio de sectores utilizables.

b. ¿Cómo se identifican las particiones en GNU/Linux? (Considere discos IDE, SCSI y SATA).

Las particiones se identifican mediante nombres de dispositivo que dependen del tipo de controlador del disco:

IDE: Los discos IDE se identifican como `/dev/hda`, `/dev/hdb`, etc. Las particiones en estos discos se numeran como `/dev/hda1`, `/dev/hda2`, y así sucesivamente.

SCSI y SATA: Los discos SCSI y SATA se identifican como `/dev/sda`, `/dev/sdb`, etc. Las particiones en estos discos se numeran como `/dev/sda1`, `/dev/sda2`, etc.

c. ¿Cuántas particiones son necesarias como mínimo para instalar GNU/Linux? Nómbralas indicando tipo de partición, identificación, tipo de File System y punto de montaje.

Para instalar GNU/Linux se requieren al menos dos particiones:

Partición raíz (/):

Tipo de partición: Primaria o lógica.

Identificación: `/dev/sda1` (ejemplo).

Tipo de File System: `ext4` (u otro sistema de archivos soportado).

Punto de montaje: `/`.

Función: Contiene todos los archivos del sistema operativo y los programas instalados.

Partición de intercambio (swap):

Tipo de partición: Primaria o lógica.

Identificación: `/dev/sda2` (ejemplo).

Tipo de File System: `swap`.

Función: Se utiliza como memoria virtual cuando la RAM física está completa. Ayuda en la gestión de procesos y mejora la estabilidad del sistema.

Opcionalmente, se puede agregar una partición para /home, que contiene los datos y configuraciones de los usuarios, permitiendo una separación clara entre los archivos del sistema y los datos personales.

d. Dar ejemplos de diversos casos de particionamiento dependiendo del tipo de tarea que se deba realizar en su sistema operativo.

1. Computadora de Escritorio de uso general (Desktop):
 - /boot (~60-100 MB, ext3/ext4): Para albergar de forma aislada el kernel y los archivos esenciales del menú de arranque.
 - / (6 GB o más, ext4): Para el núcleo del sistema y las aplicaciones instaladas.
 - /home (Espacio restante, ext4): Almacena las configuraciones y archivos personales de los usuarios (/home/usuario), permitiendo reinstalar o actualizar el sistema operativo sin perder sus datos personales.
 - SWAP: Espacio de memoria virtual de intercambio.
2. Servidor de Base de Datos o Servidor de Correo (Producción):
 - / (ext4): Partición para el arranque del sistema base.
 - /var (XFS o ext4, partición independiente): Puesto que el directorio /var almacena bases de datos, colas de correo y registros de eventos (*logs*) que varían de tamaño constantemente, aislarlo en su propia partición previene que un llenado repentino de logs sature el disco y bloquee el funcionamiento global de la partición raíz /.
 - SWAP: Memoria de intercambio para respaldar la RAM en picos de consumo de bases de datos.
3. Servidor Web con múltiples usuarios locales (Hosting):
 - / (ext4): Sistema base.
 - /home (con cuotas de disco): Para aislar los directorios de los usuarios que publican sitios web.
 - /tmp (partición independiente): Los entornos multiusuario generan archivos temporales masivos. Al separar /tmp, se evita el desbordamiento de espacio del sistema operativo principal

e. ¿Es posible visualizar particiones del tipo FAT y NTFS (que son de Windows) en GNU/Linux?

Sí, es posible visualizar y acceder a particiones del tipo FAT (FAT16, FAT32, exFAT) y NTFS en GNU/Linux

f. ¿Qué tipo de software para particionar existe? Menciónelos y compare.

Se clasifican principalmente en dos categorías según su comportamiento con los datos existentes:

1. Particionadores Destructivos:
 - Ejemplo: fdisk (herramienta interactiva por línea de comandos).
 - Funcionamiento: Permiten crear, visualizar y eliminar particiones borrando la información contenida en ellas si se altera su estructura o límites lógicos.
2. Particionadores No Destructivos:
 - Ejemplos: fips, gparted (con interfaz gráfica moderna).

- Funcionamiento: Permiten crear, eliminar y modificar (redimensionar) particiones activas sin pérdida de datos, manipulando de manera inteligente el espacio libre remanente de los volúmenes existentes. Generalmente, los instaladores de las distribuciones integran este tipo de software para facilitar el proceso gráfico durante la instalación

8. Arranque (bootstrap) de un Sistema Operativo:

a. ¿Qué es el BIOS? ¿Qué tarea realiza?

El BIOS (Basic Input/Output System) es un firmware de bajo nivel que se ejecuta al encender una computadora. Su principal tarea es inicializar y verificar el hardware del sistema, como la CPU, la RAM y los dispositivos de almacenamiento, y luego iniciar el cargador de arranque (bootloader) desde el dispositivo de arranque principal (disco duro, unidad USB, etc.) que se encarga de iniciar el sistema operativo.

b. ¿Qué es UEFI? ¿Cuál es su función?

El UEFI (Unified Extensible Firmware Interface) es el sucesor del BIOS, proporcionando una interfaz de firmware más moderna y funcional. UEFI realiza las mismas tareas que el BIOS, pero con mejoras como:

Soporte para discos de mayor tamaño (más de 2 TB).

Tiempo de arranque más rápido.

Interfaz gráfica en lugar de solo texto.

Mejor soporte de seguridad, como Secure Boot.

c. ¿Qué es el MBR? ¿Qué es el MBC?

MBR (Master Boot Record): Es el sector de arranque físico absoluto ubicado al principio del disco duro (específicamente en el cilindro 0, cabeza 0, sector 1). Su tamaño estándar de sector es de 512 bytes y contiene la siguiente información:

Los primeros 446 bytes: Reservados para el programa de arranque inicial (el MBC).

Los siguientes 64 bytes: Destinados a la tabla de particiones primarias del disco.

Los últimos 2 bytes: Utilizados como firma de validación del MBR

MBC (Master Boot Code): Es un programa ejecutable de tamaño muy reducido (máximo 446 bytes) grabado al inicio del MBR. Su función es actuar como un cargador inicial (*bootloader*) básico de una o múltiples etapas. El BIOS se encarga de leer este código, llevarlo a la memoria del sistema y ejecutarlo como paso final del proceso de inicialización

d. ¿A qué hacen referencia las siglas GPT? ¿Qué sustituye? Indique cuál es su formato.

Siglas: Significa GUID Partition Table (Tabla de Particiones de Identificadores Únicos Globales).

Sustitución: Es parte del estándar EFI y se diseñó para sustituir al antiguo esquema de particionado MBR, superando sus límites físicos e incrementando significativamente la cantidad de particiones permitidas

Formato de GPT:

Utiliza un direccionamiento lógico de bloques denominado LBA (Logical Block Addressing) en lugar del antiguo esquema CHS (*cylinder-header-sector*).

En el bloque LBA 0 guarda un MBR "heredado" (*Protective MBR*) para mantener compatibilidad con sistemas basados en BIOS tradicionales.

En el bloque LBA 1 se almacena la cabecera GPT primaria, seguida de los bloques LBA para las entradas de las particiones.

La cabecera y la tabla se guardan duplicadas tanto al principio como al final del disco para proveer redundancia ante corrupciones de datos

e. ¿Cuál es la funcionalidad de un “Gestor de Arranque”? ¿Qué tipos existen? ¿Dónde se instalan? Cite gestores de arranque conocidos.

Un gestor de arranque (bootloader) es un software responsable de cargar el sistema operativo en la memoria RAM y darle control del sistema cuando la computadora arranca. Este software se encuentra en la primera etapa del proceso de inicio, justo después de la ejecución del BIOS o UEFI.

Funcionalidades:

Cargar el kernel del sistema operativo en la memoria.

Permitir seleccionar entre diferentes sistemas operativos (en caso de sistemas dual-boot).

Pasar parámetros al kernel del sistema operativo.

Tipos de gestores de arranque:

GRUB (GRand Unified Bootloader):

Descripción: Es uno de los gestores de arranque más comunes en sistemas GNU/Linux. Soporta múltiples sistemas operativos y es altamente configurable.

Instalación: Generalmente se instala en el MBR (en sistemas BIOS) o en la partición de arranque EFI (en sistemas UEFI).

LILO (Linux LOader):

Descripción: Un gestor de arranque más antiguo que GRUB, que ha caído en desuso. Ofrece menos características y flexibilidad en comparación con GRUB.

Instalación: Se instala en el MBR o en el primer sector de una partición.

Syslinux/Extlinux:

Descripción: Un conjunto de gestores de arranque que se utiliza principalmente para medios extraíbles, como unidades USB o discos de arranque en red.

Instalación: Puede instalarse en el MBR o en una partición específica.

f. ¿Cuáles son los pasos que se suceden desde que se prende una computadora hasta que el Sistema Operativo es cargado (proceso de bootstrap)?

Secuencia general del proceso de bootstrap (Arranque)

El proceso de arranque de un sistema operativo, conocido como bootstrap, sigue estos pasos generales:

- 1: Power-On Self Test (POST): Al encender la computadora, el BIOS o UEFI realiza una serie de pruebas para verificar el hardware básico del sistema (CPU, RAM, dispositivos de almacenamiento, etc.).
- 2: Carga del firmware (BIOS/UEFI): Una vez completado el POST, el BIOS o UEFI busca el dispositivo de arranque (disco duro, unidad USB, etc.) especificado en la configuración.
- 3: Carga del gestor de arranque: El BIOS/UEFI carga el gestor de arranque desde el dispositivo de arranque. En sistemas BIOS, se lee el código del MBR; en sistemas UEFI, se busca el gestor de arranque en la partición EFI.
- 4: Selección del sistema operativo: Si hay múltiples sistemas operativos instalados, el gestor de arranque muestra un menú para seleccionar uno. Si solo hay un sistema operativo, el gestor de arranque lo carga automáticamente.
- 5: Carga del kernel: El gestor de arranque carga el kernel del sistema operativo seleccionado en la memoria y le transfiere el control.
- 6: Inicialización del kernel: El kernel inicializa el hardware del sistema, monta el sistema de archivos raíz, y lanza el proceso init o systemd, que luego inicia todos los servicios necesarios para arrancar el sistema operativo completamente.
- 7: Carga del entorno de usuario: Una vez que el kernel ha completado su inicialización, el sistema de inicio lanza los servicios de usuario y, en el caso de sistemas de escritorio, carga el entorno gráfico o la terminal de comandos.

g. Analice el proceso de arranque en GNU/Linux y describa sus principales pasos.

Una vez cargado el kernel de Linux en memoria por el cargador de arranque, ocurren los siguientes pasos:

1. Inicialización del Kernel: El núcleo Linux asume el control del procesador.
2. Detección y configuración de Hardware: El kernel comprueba, diagnostica y habilita todos los dispositivos físicos conectados a la computadora.
3. Carga de Controladores (Módulos): Para operar de forma correcta, se cargan dinámicamente los controladores necesarios a través de módulos cargables del sistema (*Loadable Kernel Modules*).
4. Montaje del sistema de archivos raíz: El kernel monta el sistema de archivos raíz (/) de forma lógica para permitir la lectura y escritura de programas.
5. Inicio de la ejecución en espacio de usuario (init): Una vez puestos a punto todos los dispositivos, el kernel transfiere el control ejecutando el primer programa del sistema, que es el proceso init. Este proceso (con PID 1) inicia ordenadamente los scripts de arranque ubicados en /etc/init.d (o servicios correspondientes) para activar la red, levantar los demonios y finalmente mostrar la pantalla de inicio de sesión (*login*)

h. ¿Cuáles son los pasos que se suceden en el proceso de parada (shutdown) de GNU/Linux? i. ¿Es posible tener en una PC GNU/Linux y otro Sistema Operativo instalado? Justifique.

El proceso de apagado de un sistema GNU/Linux sigue estos pasos:

Solicitud de apagado: El usuario o el sistema emite un comando para apagar el sistema (shutdown, halt, poweroff, etc.).

Notificación a los procesos: systemd o el sistema de inicio correspondiente notifica a todos los procesos en ejecución para que se detengan de manera ordenada. Se les envía la señal SIGTERM, permitiéndoles cerrar archivos y liberar recursos.

Detención de servicios: Todos los servicios y demonios del sistema se detienen uno por uno. Los sistemas de archivos se desmontan y los dispositivos se sincronizan.

Detención del kernel: Una vez que todos los servicios han sido detenidos y los sistemas de archivos desmontados, el kernel apaga la CPU y otros componentes del hardware, completando el proceso de apagado.

Apagado físico: Finalmente, el hardware de la máquina se apaga completamente.

9. Archivos y editores:

a. ¿Cómo se identifican los archivos en GNU/Linux?

En GNU/Linux, los archivos se identifican principalmente a través de su nombre y su ruta en el sistema de archivos. A diferencia de otros sistemas operativos, GNU/Linux no depende de las extensiones de archivo para identificar el tipo de archivo, aunque las extensiones se utilizan comúnmente para indicar el formato del archivo.

Formas de identificar archivos

Nombres de archivo y rutas:

Ruta absoluta: Especifica la ubicación completa del archivo desde el directorio raíz /. Ejemplo: /home/gianca/documento.txt.

Ruta relativa: Especifica la ubicación del archivo en relación con el directorio actual. Ejemplo: documento.txt o ../documento.txt.

Identificación por permisos y atributos:

Permisos de lectura (r): Indica si se puede leer el contenido del archivo.

Permisos de escritura (w): Indica si se puede modificar el archivo.

Permisos de ejecución (x): Indica si se puede ejecutar el archivo como un programa.

Inodos y estructura de archivos:

Inodo (i-node): Cada archivo en GNU/Linux se identifica internamente por un número de inodo, que almacena la metadata del archivo (permisos, propietario, tamaño, etc.), aunque no almacena el nombre del archivo en sí.

Los inodos permiten al sistema acceder al archivo en el sistema de archivos sin depender del nombre del archivo.

b. Investigue el funcionamiento de los editores vim, nano y mcedit, y los comandos cat, more y less.

Abrir un archivo en vi: vi archivo.txt

-

c. Cree un archivo llamado "prueba.exe" en su directorio personal usando el vim. El mismo debe contener su número de alumno y su nombre.

comando: vi prueba.exe

modo edicion: i

guardar y salir: esq, :wq

d. Investigue el funcionamiento del comando file. Pruébalo con diferentes archivos. ¿Qué diferencia nota?

file nos indica que tipo de archivo es un elemento

e. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso de archivos: i. cd ii. mkdir iii. rmdir iv. ln v. tail vi. locate vii. ls viii. pwd ix. cp x. mv xi. find

cd: change directory, sirve para cambiar de carpeta

usos: cd /ruta nos lleva a esa ruta, cd .. sube un nivel, cd ~ nos lleva a la carpeta de usuario, cd - vuelve a la carpeta anterior

mkdir: sirve para crear carpetas

usos: mkdir nuevaCarpeta crea una carpeta con ese nombre en la ubicacion actual, mkdir -p /nueva/nuevaInterna/otraNueva crea toda la ruta si es que no existe, mkdir -m 1000 carpeta le da permisos de seguridad

rmdir borra carpetas SOLO si estan vacias

usos: rmdir carpetaABorrar, -p toda la ruta

ln (LN): sirve para crear accesos directos a carpetas o archivos

usos: ln carpeta enlace, crea un enlace que apunta al mismo dato. si se borra el enlace sigue funcionando

-s: crea un enlace simbolico. Si se borra el archivo original el enlace se rompe

tail: muestra las ultimas lineas de un archivo de texto (muy util para docu o logs)

locate: te muestra la ruta de el archivo buscado

usos: locate prueba.txt muestra donde esta el archivo creado previamente, locate -i prueba.txt ignora el case, locate -n numero limita los resultados a ese numero

ls (List): listar el contenido

usos: ls muestra los archivos y carpetas en la ruta actual, -l muestra los detalles de los archivos, -a muestra los ocultos tambien, -h traduce el tamano de los archivos a formato legible para humanos

pwd: muestra la ruta absoluta hasta la ubicacion actual

pwd -P muestra la ruta si es que estamos en un enlace simbolico

cp: copiar archivos o carpetas

usos: cp origen destino hace una copia del archivo cambiandole el nombre, cp -r realiza una copia recursiva (obligatorio en caso de copiar una carpeta y su contenido) cp -r carpetaOrig carpetaCopy, -i modo interactivo, -v muestra en la terminar todo el procesos detalladamente

mv: mover archivos o carpetas

usos: mv nombrePrevio nombreNuevo lo renombra, mv archivo /ruta/destino mueve a destino el archivo, -i modo interactivo, -n no sobrescribe ningun archivo existente en el destino

find: buscar archivos con criterios avanzados (find . [parametro] -name ["nombre"])

usos: find /ruta prueba.txt busca ese archivo en esa ruta en particular, find type d busca solo por

carpetas y `find -f` busca solo archivos, `find . -size [-/+]tamano` busca archivos con mas o menos de ese tamaño, `-mtime [dias]` busca archivos modificados hace cierta cantidad de días `find . -mtime -3`

10. Indique qué comando es necesario utilizar para realizar cada una de las siguientes acciones. Investigue su funcionamiento y parámetros más importantes:

a. Cree la carpeta ISOCSO

`mkdir ISOCSO`

b. Acceda a la carpeta

`cd ISOCSO`

c. Cree dos archivos con los nombres `isocso.txt` e `isocso.csv`

`touch isocso.txt isocso.csv`

d. Liste el contenido del directorio actual

`ls`

e. Visualizar la ruta donde estoy situado

`pwd`

f. Busque todos los archivos en los que su nombre contiene la cadena `"iso"`

`find . -type f -name "*iso*"`

g. Informar la cantidad de espacio libre en disco

`df -h`

h. Verifique los usuarios conectados al sistema

`who`

i. Editar a el archivo `isocso.txt` e ingresar Nombre y Apellido

-

j. Mostrar en pantalla las últimas líneas de un archivo.

`tail isocso.txt`

11. Investigue el objetivo, parámetros y ubicación (directorio) de los siguientes comandos: → `man` → `shutdown` → `reboot` → `halt` → `uname` → `dmesg` → `lspci` → `at` → `head` → `tail`

man: manual de ayuda, ubicado en /usr/bin/man

Parametros: man [comando] muestra el manual de ese comando, -k [string] muestra comandos relacionados a ese string

shutdown: apagar, reiniciar o programar un apagado, ubicado en /usr/sbin/shutdown

Parametros: -h apaga el equipo por completo, -r reinicia el equipo, now o +tiempo define cuando se apaga (shutdown -h +20 (se apagara en 20 minutos)), -c cancela el apagado en proceso

reboot: reiniciar, ubicacion: /usr/sbin/reboot

Parametros: -f fuerza el reinicio inmediatamente

halt: detiene todas las funciones del cpu y congela el sistema, ubicacion: /usr/sbin/halt

Parametros: -p detiene el sistema y corta la energia, -f fuerza la detencion sin pasar por el proceso normal de cierre

uname: imprime la informacion del sistema operativo, nombre del equipo, kernel y detalles del nucleo, ubicacion: /usr/bin/uname

Parametros: -a muestra toda la info disponible, -r muestra solo el kernel, -m muestra la arquitectura

dmesg: muestra los mensajes del buffer del anillo del kernel. sirve para ver que controladores, discos o componentes se cargaron o fallaron recientemente, ubicacion /usr/bin/dmesg

Parametros: -T muestra los mensajes con horas legibles, -l (L) [nivel] filtra segun la gravedad (ej: err)

lspci: lista y detalla todos los dispositivos conectados a los PCI, ubicacion: /usr/bin/lspci

parametros: -v modo detallado, -nn muestra la descripcion y sus codigos de fabricante

at: permite programar la ejecucion de un comando o tarea de unica vez en un futuro, ubicacion: /usr/bin/at

Parametros: at [22:00], at now +2 hours, -l lista todas las tareas programadas pendientes, -r [id] elimina esa tarea

head: muestra las primeras lineas de un archivo, ubicacion: /usr/bin/head. Parametros: -n numero muestra esa cantidad de lineas

tail: muestra las ultimas lineas de un archivo, ubicacion: /usr/bin/tail. Parametros: -n numero muestra esa cantidad de lineas, -f mantiene el archivo abierto para ver en tiempo real

12. Proceso de Arranque SystemV :

a. Enumere los pasos del proceso de inicio de un sistema GNU/Linux, desde que se prende la PC hasta que se logra obtener el login en el sistema.

El viaje desde que presionas el botón de encendido hasta la pantalla de inicio de sesión consta de 6 etapas principales:

1. BIOS / UEFI: La placa madre se enciende y ejecuta el *POST* (Power-On Self-Test) para verificar el hardware (RAM, discos, teclado). Luego, busca en el orden de arranque el dispositivo que tenga el gestor de inicio.
2. Gestor de Arranque (Bootloader - usualmente GRUB): El BIOS/UEFI carga GRUB en la memoria. GRUB muestra el menú de selección de sistemas operativos y le pasa al procesador los parámetros necesarios para cargar el núcleo de Linux.
3. Kernel (Núcleo): El Kernel de Linux se monta en la memoria, detecta y configura todos los componentes de hardware restantes y monta el sistema de archivos raíz (/) en modo de solo lectura.
4. Proceso INIT: Una vez establecido el entorno básico, el Kernel arranca el primer programa del espacio de usuario: el proceso init (ubicado en /sbin/init).
5. Scripts de los Niveles de Ejecución (Runlevels): El proceso init lee su configuración, determina el nivel de ejecución asignado y arranca en orden todos los servicios del sistema (red, bases de datos, sonido, etc.).
6. Pantalla de Login: Finalmente, init arranca el proceso getty (para consolas de texto TTY) o un Gestor de Pantalla visual (como GDM o LightDM) para pedir el usuario y la contraseña.

b. Proceso INIT. ¿Quién lo ejecuta? ¿Cuál es su objetivo?

Es ejecutado por el kernel de linux al finalizar su propia carga

Objetivo: Su meta es inicializar el espacio de usuario

c. RunLevels. ¿Qué son? ¿Cuál es su objetivo?

Son estados o modos de operación en los que se encuentra un sistema operativo basado en Unix/Linux. Cada nivel define qué programas y servicios tienen permitido ejecutarse en ese momento

Su objetivo es permitir al administrador controlar el comportamiento del sistema

d. ¿A qué hace referencia cada nivel de ejecución según el estándar? ¿Dónde se define qué Runlevel ejecutar al iniciar el sistema operativo? ¿Todas las distribuciones respetan estos estándares?

0: apagado, se usa para cerrar el sistema de forma segura

1: modo monousuario (sin red ni entorno grafico, inicia sesion directo como admin)

2: modo multiusuario sin red

3: modo multiusuario completo en consola (interfaz pura de textp)

4: Undefined, permite crear un entorno a medida

5: multiusuario completo grafico

6: reinicio

El runlevel se definen mediante enlacez simbolicos como graphical.target o multi-user.target

No todas de la misma manera. Mientras que RedHat, Fedora y CentOS usaban históricamente el estándar estricto (3 para texto, 5 para gráfico), distribuciones como Debian y Ubuntu configuraban los niveles del 2 al 5 exactamente de la misma manera: como entornos multiusuario completos con red y gráficos activados si estaban instalados. Además, hoy en día la mayoría de las distribuciones modernas migraron de SystemV a systemd.

e. Archivo `/etc/inittab`. ¿Cuál es su finalidad? ¿Qué tipo de información se almacena en él? ¿Cuál es la estructura de la información que en él se almacena?

Es el archivo principal de configuración del proceso init, le dice al sistema como arrancar. Se almacena el runlevel default, que scripts correr, que hacer al presionar `ctrl alt supr`, y cuantas terminales de texto abrir.

Estructura: 4 campos separados por `:` ej: `id:2:initdefault:`

f. Suponga que se encuentra en el runlevel 2. Indique qué comando(s) deberá ejecutar para cambiar al runlevel 5. ¿Este cambio es permanente? ¿Por qué?

`sudo init 5`, este cambio no es permanente ya que en el init se lee el runlevel por default.

g. Scripts RC. ¿Cuál es su finalidad? ¿Dónde se almacenan? Cuando un sistema GNU/Linux arranca o se detiene se ejecutan scripts, indique cómo determina qué script ejecutar ante cada acción. ¿Existe un orden para llamarlos? Justifique.

Son scripts escritos en shell encargados de arrancar o detener servicios individuales.

Ubicación: `/etc/init.d/` y específicas de nivel: `/etc/rc1.d/`, `/etc/rc2.d`

Dentro de las carpetas están los enlaces simbólicos a los scripts maestros, el sistema determina que hacer leyendo la primera letra del enlace: `k` para apagar y `s` para iniciar. El orden depende del número posterior a la letra `k` o `s`.

13. SystemD (<https://github.com/systemd/systemd>):

a. ¿Qué es systemd?

Es un administrador de sistema y servicios que actúa como PID 1 para iniciar y supervisar procesos.

b. ¿A qué hace referencia el concepto de Unit en SystemD?

Units o unidades definen recursos como servicios o montajes.

c. ¿Para qué sirve el comando `systemctl` en SystemD?

Es el comando principal para gestionar los targets.

d. ¿A qué hace referencia el concepto de target en SystemD?

Los targets agrupan unidades para establecer estados específicos.

e. Ejecutar el comando pstree. ¿Qué es lo que se puede observar a partir de la ejecución de este comando?

Muestra la jerarquia completa de procesos

14. Usuarios:

a. ¿Qué archivos son utilizados en un sistema GNU/Linux para guardar la información de los usuarios?

Se utilizan principalmente 4 archivos de texto plano ubicados en /etc

/etc/passwd guarda la informacion basica de los usuarios

/etc/shadow guarda las contraseñas cifradas

/etc/group contiene la lista de los grupos del sistema y los usuarios que pertenecen a cada uno

/etc/gshadow guarda las contraseñas cifradas de los grupos

b. ¿A qué hacen referencia las siglas UID y GID? ¿Pueden coexistir UIDs iguales en un sistema GNU/Linux? Justifique.

UID Numero de identificacion de usuario

GID Numero de identificacion de grupo

Pueden existir dos UID iguales, pero serán el mismo usuario con dos alias diferentes

c. ¿Qué es el usuario root? ¿Puede existir más de un usuario con este perfil en GNU/Linux? ¿Cuál es la UID de root?

Es el superusuario o admin. Es una cuenta especial que posee control total del OS. Su UID es siempre 0. Puede existir mas de un usuario unicamente duplicando si UID en el archivo etc passwd

d. Agregue un nuevo usuario llamado isocso a su instalación de GNU/Linux, especifique que su home sea creada en /home/isocso, y hágalo miembro del grupo informatica (si no existe, deberá crearlo). Luego, sin iniciar sesión como este usuario cree un archivo en su home personal que le pertenezca. Luego de todo esto, borre el usuario y verifique que no queden registros de él en los archivos de información de los usuarios y grupos.

```
sudo groupadd informatica
```

```
sudo useradd -m -d /home/isocso -g isocso
```

```
sudo useradd -m -d /home/isocso -g informatica isocso
```

```
sudo touch /home/isocso/archivo_prueba.txt
```

```
sudo chown isocso:informatica /home/isocso/archivo_prueba.txt
```

```
sudo userdel -r isocso
```

```
grep isocso /etc/passwd
```

```
grep isocso /etc/group
```

```
grep isocso /etc/shadow
```

e. Investigue la funcionalidad y parámetros de los siguientes comandos: i. useradd y adduser ii. usermod iii. userdel iv. su v. groupadd vi. who vii. groupdel viii. passwd

useradd y adduser: permiten crear cuentas de usuario nuevas en el sistema con la diferencia en su nivel, useradd es de bajo nivel y adduser es de alto nivel

Parametros: -m crea el directorio personal

-d [ruta] crea una ruta personalizada

-g [grupo] asigna el grupo del usuario

-G [grupos] asigna grupos secundarios

-s [shell] define la terminar por defecto

usermod: modifica los atributos de una cuenta de usuario existente

-l [nuevo nombre] cambia el nombre de login

-d [nueva ruta] modifica la ruta del directorio personal

-g [nuevo grupo] modifica el grupo principal

-aG [grupo] le agrega al usuario un grupo suplementario

-L / -U bloquea o desbloquea la contraseña del usuario

userdel: elimina una cuenta de usuario

Parametros: -r borra tambien el directorio personal de usuario (recomendado)

su (switch user): permite cambiar temporalmente de usuario en la terminar actual

parametros: - (o -l / --login) simula un inicio de sesion completo, -c [comando] ejecuta un unico comando con los permisos del usuario especificado

groupadd: crea un nuevo grupo

Parametros: -g [GID] asigna un id especifico, -r crea un grupo del sistema con un gid menor

who: muestra una lista de los suarios que se encuentran actualmente conectados

Parametros: -H muestra una cabecera con el titulo de cada columna, -u muestra informacion adicional como el tiempo de inactividad, am i: muestra el usuario actual

groupdel: elimina un grupo

Parametros: [nombre grupo]

passwd: cambiar la contraseña de una cuenta de usuario, un usuario comun solo puede cambiar la propia

Parametros: -d elimina la contraseña, -l / -u bloquea o debloquea, -e caduca la contraseña y debera cambiarla

15. FileSystem y permisos:

a. ¿Cómo son definidos los permisos sobre archivos en un sistema GNU/Linux?

Los permisos se definen dividiendo a los usuarios en 3 categorias: Dueño: u, Grupo: g, Otros: o

Con 3 tipos de permisos: lectura: r, escritura: w, ejecucion: x

b. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con los permisos en GNU/Linux: i. chmod ii. chown iii. chgrp

chmod: modifica los permisos de acceso de un archivo o directorio

Parametros: -R recursivo para borrar todo el contenido

chown: cambia el dueño de un archivo

Parametros: -R ej: chown lucas:informatica prueba.txt

chgrp: cambia el grupo propietario de un archivo

Parametros: -R

c. Al utilizar el comando chmod generalmente se utiliza una notación octal asociada para definir permisos. ¿Qué significa esto? ¿A qué hace referencia cada valor?

Utiliza un numero de 3 digitos que representan los permisos de dueño, grupo, otros, cada valor surge de la suma de los permisos:

lectura= 4

escritura=2

ejecucion=1

Ej: chmod 750 arch significa:

7 (4+2+1) para el dueño lectura escritura y ejecucion

5 (4+1) para el grupo lectura y ejecucion

0 para otros: sin permisos

d. ¿Existe la posibilidad de que algún usuario del sistema pueda acceder a determinado archivo para el cual no posee permisos? Indíquelo y realice las pruebas correspondientes.

Si, el usuario root tiene acceso a todo sin importar los permisos

e. Explique los conceptos de "full path name" (path absoluto) y "relative path name" (path relativo). De ejemplos claros de cada uno de ellos.

full path es la ruta completa desde el inicio /

relative path es la ruta desde esta posición a la ruta destino

ej: /home/lucas/descargas/prueba.txt

ej: (desde /home/lucas)/descargas/prueba.txt

f. ¿Con qué comando puede determinar en qué directorio se encuentra actualmente?

Con pwd

¿Existe alguna forma de ingresar a su directorio personal sin necesidad de escribir todo el path completo?

con cd ~, cd o \$HOME

¿Podría utilizar la misma idea para acceder a otros directorios? ¿Cómo? Explique con un ejemplo.

con `cd ..` se sube un nivel, se puede usar repetidas veces

g. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso del FileSystem: i. `umount` ii. `du` iii. `df` iv. `mount` v. `mkfs` vi. `fdisk` (con cuidado) vii. `write` viii. `losetup` ix. `stat`

`umount`: desmonta un sistema de archivos accesible en el árbol de directorios

`du`: estima y muestra el espacio total, usado y disponible que consumen los archivos y directorios específicos

Parámetros: `-sh` resumido y legible para humanos

`df`: muestra el espacio total, usado y disponible en los sistemas de archivos montados

Parámetros: `-h` legible para humanos

`mount`: conecta un dispositivo de almacenamiento en un directorio específico del FS

Parámetros: `-t [type]`

`mkfs`: crea un nuevo sistema de archivos (formatea) en una partición específica

Parámetros: `-t ext4` para definir el formato

`disk`: manipulador de tablas de particiones de discos

Parámetros: `-l` para listar todas las particiones del sistema

`write`: envía un mensaje directo a la terminal de otro usuario conectado en el mismo sistema

`losetup`: configura y gestiona dispositivos de bucle, que permiten montar archivos contenedores (como las iso) como si fueran discos físicos

`stat`: muestra información detallada y de bajo nivel sobre el estado de un archivo o un sistema de archivos

16. Procesos:

a. ¿Qué significa que un proceso se está ejecutando en Background? ¿Y en Foreground?

Foreground (primer plano): el proceso se ejecuta interactuando directamente con la terminal.

Bloquea la entrada de comandos hasta que finalice o sea interrumpido

Background (segundo plano): El proceso se ejecuta de manera oculta o en paralelo, permitiendo tener la terminal libre para otros comandos

b. ¿Cómo puedo hacer para ejecutar un proceso en Background? ¿Como puedo hacer para pasar un proceso de background a foreground y viceversa?

Ejecutar en background: & al final del comando

Pasar de back a fore: fg (esto trae el ultimo proceso en segundo plano al primer plano). Si hay varios se usa fg %numero

Pasar de fore a back: Primero se suspende el proceso con ctrl + Z y luego se lo manda a segundo plano con bg

c. Pipe (|). ¿Cuál es su finalidad? Cite ejemplos de su utilización.

Su finalidad es conectar la salida estandar (stdout) de un comando directamente con la entrada estandar (stdin) de otro, permitiendo encadenar operaciones. Ej: ls -l | grep "text" para listar y mostrar solo los txt

d. Redirección. ¿Qué tipo de redirecciones existen? ¿Cuál es su finalidad? Cite ejemplos de utilización.

Permiten desviar los flujos de e/s predeterminados de los comandos

Los principales son:

>: redirige y sobrescribe la salida estandar de un archivo

>>: redirige y anade la salida al final de un archivo existente sin borrar su contenido previo. Ej: echo "hola" >> archivo.txt

< redirige la entrada estandar desde unarchivo en lugar del teclado. Ej: wc -l < archivo.txt

17. Otros comandos de Linux (Indique funcionalidad y parámetros):

a. ¿A qué hace referencia el concepto de empaquetar archivos en GNU/Linux?

Significa agrupar multiples archivos y directorios en un unico archivo contenedor(tarbal) sin comprimirlos ni reducir su tamaño

b. Seleccione 4 archivos dentro de algún directorio al que tenga permiso y sume el tamaño de cada uno de estos archivos. Cree un archivo empaquetado conteniendo estos 4 archivos y compare los tamaños de los mismos. ¿Qué característica nota?

Si sumás el tamaño individual de 4 archivos y luego medís el archivo empaquetado resultante, notarás que el archivo empaquetado pesa ligeramente un poco más que la suma de los cuatro. Esto ocurre porque el empaquetador añade una pequeña cabecera de metadatos por cada archivo para saber cómo separarlos y restaurarlos después

c. ¿Qué acciones debe llevar a cabo para comprimir 4 archivos en uno solo? Indique la secuencia de comandos ejecutados.

tar -czvf archivo_comprimido.tar.gz archivo1 archivo2 archivo3 archivo4

d. ¿Pueden comprimirse un conjunto de archivos utilizando un único comando?

Sí, utilizando herramientas como tar con la opción de compresión (-z para gzip o -j para bzip2) o utilizando directamente gzip archivo1 archivo2 archivo3 archivo4 (aunque gzip compila cada uno por separado a menos que se use un contenedor común).

e. Investigue la funcionalidad de los siguientes comandos: i. tar

Empaqueta y desempaqueta archivos. Parametros: -c crear, -x extraer, -v detallar, -f archivo, -z comprimir con gzip

ii. grep

Busca patrones de texto o expresiones regulares dentro de archivos

iii. gzip / unzip

comprime archivos reduciendo su tamaño / descomprime

iv. zgrep

permite buscar texto dentro de archivos comprimidos

v. wc

cuenta líneas, palabras y caracteres de un archivo. Parametro: -l para contar solo líneas

18. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando, indique la razón:

ls -l > prueba

ps > PRUEBA chmod 710 prueba

chown root:root PRUEBA

chmod 777 PRUEBA

chmod 700 /etc/passwd

passwd root

rm PRUEBA

man /etc/shadow : permiso denegado

find / -name *.conf

usermod root -d /home/newroot -L : permiso denegado

cd /root : permiso denegado

rm * :directorios no pueden borrarse y para carpetas principales del sistema no se posee permiso

cd / etc

cp * /home -R :permiso denegado

shutdown

19. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

a. Terminar el proceso con PID 23.

kill 23

b. Terminar el proceso llamado init o systemd. ¿Qué resultados obtuvo?

kill 1 (no lo permite)

c. Buscar todos los archivos de usuarios en los que su nombre contiene la cadena ".conf"

find /home -name "*.conf"

d. Guardar una lista de procesos en ejecución el archivo /home/procesos

ps aux > /home/usuario/procesos

e. Cambiar los permisos del archivo /home//xxxx a: i. Usuario: Lectura, escritura, ejecución ii. Grupo: Lectura, ejecución iii. Otros: ejecución

chmod 751 /home/lucas/ISOCSO/xxxx

f. Cambiar los permisos del archivo /home//yyyy a: i. Usuario: Lectura, escritura. ii. Grupo: Lectura, ejecución iii. Otros: Ninguno

chmod 650 /home/lucas/ISOCSO/yyyy

g. Borrar todos los archivos del directorio /tmp

rm -rf /tmp/*

h. Cambiar el propietario del archivo /opt/isodata al usuario isocso

sudo chown isocso /opt/isodata

i. Guardar en el archivo /home//donde el directorio donde me encuentro en este momento, en caso de que el archivo exista no se debe eliminar su contenido anterior.

pwd >> /home/lucas/donde

20. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

a. Ingrese al sistema como usuario "root"

su -

b. Cree un usuario. Elija como nombre, por convención, la primera letra de su nombre seguida de su apellido. Asígnele una contraseña de acceso.

adduser lsilva

c. ¿Qué archivos fueron modificados luego de crear el usuario y qué directorios se crearon?
/etc/passwd

/etc/shadow

/etc/group

/etc/gshadow

Se crea automáticamente el directorio personal del usuario en /home/lsilva

Se copian los archivos de configuración iniciales por defecto desde el directorio plantilla /etc/skel/ hacia dentro de /home/lsilva

d. Crear un directorio en /tmp llamado miCursada

mkdir /tmp/miCursada

e. Copiar todos los archivos de /var/log al directorio antes creado.

cp -r /var/log/* /tmp/miCursada/

f. Para el directorio antes creado (y los archivos y subdirectorios contenidos en él) cambiar el propietario y grupo al usuario creado y grupo users.

chown -R lsilva:users /tmp/miCursada

g. Agregue permiso total al dueño, de escritura al grupo y escritura y ejecución a todos los demás usuarios para todos los archivos dentro de un directorio en forma recursiva.

chmod -R 723 /tmp/miCursada

h. Acceda a otra terminal para loguearse con el usuario antes creado.

su -lsilva

i. Una vez logueado con el usuario antes creado, averigüe cuál es el nombre de su terminal.

2

j. Verifique la cantidad de procesos activos que hay en el sistema.

ps -e | wc -l (172)

k. Verifique la cantidad de usuarios conectados al sistema.

```
who | wc -l (2)
```

l. Vuelva a la terminal del usuario root y envíele un mensaje al usuario anteriormente creado enviándole que el sistema va a ser apagado.

```
write lsilva pts/1
```

m. Apague el sistema.

```
shutdown -h now
```

21. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

n. Cree un directorio cuyo nombre sea su número de legajo e ingrese a él.

```
mkdir 24461
```

```
cd 24461
```

o. Cree un archivo utilizando el editor de textos vi, e introduzca su información personal: Nombre, Apellido, Número de alumno y dirección de correo electrónico. El archivo debe llamarse "LEAME".

```
vi LEAME
```

```
ingresamos los datos
```

```
esc
```

```
:wq
```

p. Cambie los permisos del archivo LEAME, de manera que se puedan ver reflejados los siguientes permisos: ➤ Dueño: ningún permiso ➤ Grupo: permiso de ejecución ➤ Otros: todos los permisos

```
chmod 017 LEAME
```

q. Vaya al directorio /etc y verifique su contenido. Cree un archivo dentro de su directorio personal cuyo nombre sea leame donde el contenido del mismo sea el listado de todos los archivos y directorios contenidos en /etc. ¿Cuál es la razón por la cuál puede crear este archivo si ya existe un archivo llamado "LEAME" en este directorio?

```
cd /etc
```

```
ls -la
```

```
ls -la /etc > ~/24461/leamecd /etc
```

```
ls -la
```

```
ls -la /etc > ~/24461/leame
```

Se puede porque linux es case sensitive

r. ¿Qué comando utilizaría y de qué manera si tuviera que localizar un archivo dentro del filesystem? ¿Y si tuviera que localizar varios archivos con características similares? Explique el

concepto teórico y ejemplifique.

Teoría: find busca en tiempo real recorriendo la estructura de directorios del sistema de archivos según los criterios dados. locate consulta una base de datos interna indexada previamente, lo que lo hace mucho más rápido.

```
find / -name "archivo.txt" 2>/dev/null
```

Para localizar varios archivos con características similares (comodines o patrones): Se utilizan expresiones regulares o comodines (*wildcards*) como el asterisco (*) o el signo de interrogación (?).

Ejemplo (buscar todos los archivos que comiencen con "doc" y terminen en ".pdf"):

```
find /home -name "doc*.pdf" 2>/dev/null
```

s. Utilizando los conceptos aprendidos en el punto anterior, busque todos los archivos cuya extensión sea .so y almacene el resultado de esta búsqueda en un archivo dentro del directorio creado en el primer inciso. El archivo deberá llamarse ejercicioF.

```
find / -name "*.so" > ~/24461/ejercicioF 2>/dev/null
```

22. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando indique la razón:

01. mkdir iso se crea la carpeta

02. cd . / iso; ps > f0 demasiados argumentos (y espacios)

03. ls > f1 crea un dir con el listado de archivos actual

04. cd / change directory a la raiz

05. echo \$HOME muestra el valor de la variable de entorno home, es decir la ruta del dir al directorio personal de user

06. ls -l \$> \$HOME/ iso/ls el uso de >\$> es invalido (y hay un espacio)

07. cd \$HOME; mkdir f2 se mueve al directorio personal y crea el directorio f2

08. ls -ld f2 muestra los detalles de ese directorio

09. chmod 341 f2 cambia los permisos de f2 a dueno: 3 (escribir(2) ejecutar(1)) grupo 4 (leer(4)) otros 1 (ejecutar(1))

10. touch dir crea un archivo llamado dir

11. `cd f2` se mueve a f2

12. `cd ~/iso` se mueve a iso

13. `pwd > f3` guarda la ruta del dir actual en un directorio f3

14. `ps | grep 'ps' | wc -l >> ../f2/f3` Lista los procesos, filtra cuantas veces aparece la palabra 'ps', cuenta las líneas y anade (>>) ese número al final del archivo f3 ubicado un nivel arriba en f2

15. `chmod 700 ../f2 ; cd ..` cambia los permisos de f2 con acceso total SOLO al dueño y posteriormente se mueve un nivel arriba

16. `find . -name etc/passwd` hacia abajo pero no encuentra nada

17. `find / -name etc/passwd` busca en todo el directorio pero se le deniega el acceso

18. `mkdir ejercicio5` crea ese directorio en el /

19 `cp -r iso 24461/`

20 `cp explicacion 24461/`

22. Inicie 2 sesiones utilizando su nombre de usuario y contraseña. En una sesión vaya siguiendo paso a paso las órdenes que se encuentran escritas en el cuadro superior. En la otra sesión, cree utilizando algún editor de textos un archivo que se llame "explicacion_de_ejercicio" dentro del directorio creado en el ejercicio 22 y, para cada una de los comandos que ejecute en la otra sesión, realice una breve explicación de los resultados obtenidos.

b. Complete los comandos 19 y 20, de manera tal que realicen la siguiente acción: 19: Copiar el directorio iso y todo su contenido al directorio creado en 24.a (ej 22.a) 20: Copiar el resto de los archivos y directorios que se crearon en este ejercicio al directorio creado en el ejercicio 24.a

c. Ejecute las órdenes 19 y 20 y coméntelas en el archivo creado en el inciso a).

23. Cree una estructura desde el directorio /home que incluya varios directorios, subdirectorios y archivos, según el esquema siguiente. Asuma que "usuario" indica cuál es su nombre de usuario. Además deberá tener en cuenta que dirX hace referencia a directorios y fX hace referencia a archivos. Utilizando la estructura de directorios anteriormente creada, indique qué comandos son necesarios para realizar las siguientes acciones:

a. Mueva el archivo "f3" al directorio de trabajo /home/usuario.

`mv f3 ~`

b. Copie el archivo "f4" en el directorio "dir11".

`cp -r f4 dir11/`

c. Haga lo mismo que en el inciso anterior pero el archivo de destino, se debe llamar "f7".

```
cp -r f4 dir11/f7
```

d. Cree el directorio copia dentro del directorio usuario y copie en él, el contenido de "dir1". (dir 11?)

```
mkdir ~/copia
```

```
cp -r dir11/* ~/copia/
```

e. Renombre el archivo "f1" por el nombre archivo y vea los permisos del mismo.

```
mv f1 archivo && ls -l archivo
```

f. Cambie los permisos del archivo llamado archivo de manera de reflejar lo siguiente: ➤

Usuario: Permisos de lectura y escritura ➤ Grupo: Permisos de ejecución ➤ Otros: Todos los permisos

```
chmod 617 archivo
```

(6 es 4 de lectura y 2 escritura) (1 de ejecutar) (7 de los 3 permisos)

g. Renombre los archivos "f3" y "f4" de manera que se llamen "f3.exe" y "f4.exe" respectivamente.

```
mv f3 f3.exe
```

```
mv f4 f4.exe
```

h. Utilizando un único comando cambie los permisos de los dos archivos renombrados en el inciso anterior, de manera de reflejar lo siguiente: ➤ Usuario: Ningún permiso ➤ Grupo:

Permisos de escritura ➤ Otros: Permisos de escritura y ejecución

```
chmod 023 f3.exe f4.exe (0 ninguno, 2 escritura, 3 de escritura y ejecución)
```

24. Indique qué comando/s es necesario para realizar cada una de las acciones de la siguiente secuencia de pasos (considerando su orden de aparición):

a. Cree un directorio llamado logs en el directorio /tmp.

```
cd tmp
```

```
mkdir logs
```

b. Copie todo el contenido del directorio /var/log en el directorio creado en el punto anterior.

```
cp -r /var/log/* /tmp/logs/ (permisos denegados en algunos archivos)
```

c. Empaquete el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar".

hay que moverse a la carpeta para evitar el error de permisos:

```
cd /tmp
```

```
tar -cvf ~/misLogs.tar logscd /tmp
```

```
tar -cvf ~/misLogs.tar logs
```

o ir con sudo

```
sudo tar -cvf misLogs.tar /tmp/logs
```

d. Empaquete y comprima el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar.gz".

```
tar -czvf misLogs.tar.gz /tmp/logs
```

e. Copie los archivos creados en c) y d) al directorio de trabajo de su usuario.

```
cp misLogs.tar misLogs.tar.gz ~/
```

f. Elimine el directorio creado en a), logs.

```
rm -rf /tmp/logs
```

g. Desempaquete los archivos creados en c y d en 2 directorios diferentes.

x le dice que extraiga

z descomprime los que terminan en gz

v modo detallado

f especifica el archivo

-C le dice a donde moverse antes de descomprimir

```
mkdir dir_tar dir_gz
```

```
tar -xvf misLogs.tar -C dir_tar
```

```
tar -xzvf misLogs.tar.gz -C dir_gz
```